

Received 11 June 2025, accepted 22 June 2025, date of publication 30 June 2025, date of current version 8 July 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3584060

RESEARCH ARTICLE

A Systematic Literature Review on Large Language Models Applications in Computer Programming Teaching Evaluation Process

ANDRE FABIANO PEREIRA^{ID} AND RAFAEL FERREIRA MELLO^{ID}, (Member, IEEE)

Centro de Estudos e Sistemas Avançados do Recife, Recife, Pernambuco 50030-390, Brazil

Corresponding author: Andre Fabiano Pereira (afsp@cesar.school)

This work was supported in part by the Centro de Estudos e Sistemas Avançados do Recife.

ABSTRACT Tools based on the use of Large Language Models (LLMs) have improved the computer programming teaching process, automated feedback processes, facilitated program repair, and enabled personalized learning experiences. This research examines which and how LLM-based opportunities are applied in the computer programming teaching assessment process and how LLMs are applied to improve evaluation accuracy, their impact on student learning outcomes, and the challenges in scaling these technologies. Key opportunities arise from prompt engineering, which optimizes precision and LLM-generated feedback relevance, and feedback propagation techniques, which offer scalable solutions for large-scale programming courses. LLMs are also applied effectively in debugging assistance to detect and repair syntactic and semantic errors in student code. This review identifies several research directions, including prompt engineering refinement, improved feedback system scalability, and deeper exploration of the long-term educational impacts of LLM. The study concludes that LLMs are effective in enhancing the assessment process, but a balanced approach combining human oversight with automated feedback is crucial to fostering critical thinking and ensuring long-term learning success in programming education.

INDEX TERMS Programming education, assessment processes, large language models, adaptive learning.

I. INTRODUCTION

The Large Language Models (LLMs) in computer programming education have received significant attention in recent years due to their ability to generate human-like text, provide timely feedback, and interact with students adaptively [1], [2].

This work aims to construct a systematic literature review (SLR), synthesizing the available evidence to address questions about the use of LLM to support and improve assessment processes in computer programming education, with the aim of integrating results from empirical studies and providing information on applied methodologies, evaluation metrics, and pedagogical implications [3], [4].

This SLR focuses specifically on identifying how LLMs have been applied to support teaching and evaluation processes in programming education. Unlike general surveys,

this study systematically compiles and analyzes empirical evidence related to the effectiveness, limitations, and scalability of LLM-based educational interventions.

Are the objectives of the Review: (1) Identify the main uses of LLMs in the context of programming education, (2) evaluate the benefits and limitations of LLMs in supporting learning and assessment tasks, and (3) synthesize implications for instructors and researchers who wish to implement LLMs in their teaching practices.

Traditional programming education faces challenges in providing consistent and high-quality evaluation and feedback to students, particularly in large courses where the student-instructor ratio makes individualized attention difficult [5]. Manual grading methods often result in delays that reduce student motivation and participation. LLMs offer a solution by automating the feedback process and enabling scalable and real-time responses [6]. These tools can help students identify and correct errors, improving

The associate editor coordinating the review of this manuscript and approving it for publication was Jeffrey Jonathan Yackley^{ID}.

their learning experience while reducing the workload of educators [7].

LLMs, such as GPT-4 and Codex [8], can modify feedback processes by allowing for more immediate and relevant responses to student errors [9]. Their ability to assist with program repair and provide personalized learning experiences has made them a solution to improve student engagement and learning outcomes [3], [10].

However, despite these advances, there are still limitations in manual evaluation processes, which this review aims to address by examining how LLM-based tools can improve feedback accuracy, increase student engagement, and lead to better learning outcomes [3], [10].

Several studies have highlighted the effectiveness of LLM in programming education. For example, [8] demonstrated that Codex, a GPT-4 version fine-tuned for code, was more efficient in detecting and repairing syntactic and semantic bugs than traditional methods. Similarly, [11] showed that students using InferFix, an LLM-based program repair tool, completed tasks 20% faster than those relying on manual debugging.

These findings suggest that LLMs have the potential to significantly improve both the efficiency and effectiveness of programming education [12], [13].

Beyond feedback and program repair, LLMs are being applied to personalized learning systems. Reference [14] developed the Tutorly system, which transformed static programming tutorials into interactive learning environments, allowing students to receive real-time feedback tailored to their needs.

This adaptability ensures that students with varying skill levels are supported, particularly in large-scale courses where individualized attention is scarce [4].

Despite these promising results, several challenges remain. LLMs must be fine-tuned to avoid overreliance on automated solutions and ensure that they foster long-term problem solving skills in students [15]. Furthermore, scalability and efficient use of computational resources are key areas that require further exploration [13].

This study contributes to the field by (i) presenting a systematic classification of llm-based evaluation methodologies, based on scope, automation level, and pedagogical alignment [3]; (ii) defining a set of quantitative and qualitative assessment metrics, consolidating key performance indicators such as accuracy, response latency, scalability, and student engagement to standardize the evaluation of llm-generated feedback [8], [14], [16]; and (iii) providing a critical discussion on LLM limitations in automated assessment, highlighting challenges such as hallucinations, ethical concerns, and the necessity of human oversight [17], [18], [19].

This review identifies research directions such as prompt engineering refinement, improvements in scalability, and a deeper exploration of the long-term educational impacts of LLMs [1].

II. THEORETICAL BACKGROUND

LLMs, such as OpenAI's GPT-4, are trained on extensive datasets and have demonstrated impressive abilities in both understanding and generating codes [20]. These models have proven to be effective in providing real-time feedback, helping to debug and supporting personalized learning, making them an essential tool in modern programming education [2], [11], [12].

This section reviews the theoretical underpinnings of LLMs in programming education and highlights their contributions to feedback generation, personalized learning, debugging, and program repair. In addition, it identifies existing gaps in the literature and outlines opportunities for future research.

A. LLMs IN EDUCATIONAL CONTEXTS

LLMs have been increasingly used in educational environments because of their ability to process and generate language in a relevant way. In programming education, LLMs are used to automate feedback processes, allowing real-time responses to student submissions [10]. For example, [8] demonstrated how Codex, an LLM specifically trained in coding tasks, improved students' ability to debug and repair code. The ability to provide instant feedback has been found to improve student learning outcomes, particularly in large-scale courses where manual feedback is often delayed [3].

B. AUTOMATED FEEDBACK MECHANISMS

The potential of LLM to automate feedback mechanisms has been a major focus of research in programming education. Traditional feedback methods are labor intensive and can suffer from inconsistencies owing to human bias [15]. By contrast, LLMs offer scalable and consistent feedback across a wide range of programming tasks. Studies such as those by [4] and [9] have demonstrated that LLM-powered tools reduce instructors' workload and provide more timely and accurate feedback to students.

Automated feedback systems powered by LLMs have also been shown to promote active learning by allowing students to iteratively correct their mistakes based on real-time feedback [13]. This iterative process has been particularly effective in improving problem-solving skills and fostering independent learning, as students are encouraged to rely less on instructor guidance and more on LLM feedback [11], [14].

C. PROGRAM REPAIR AND DEBUGGING ASSISTANCE

LLM have proven to be particularly useful in the area of program repair and debugging. Tools such as InferFix and PyDex, which use LLMs for automated bug detection and code repair, have demonstrated superior performance in identifying and fixing syntactic and semantic errors compared to traditional debugging tools [8], [11]. These tools save time and improve the accuracy of repairs, providing

students with immediate actionable feedback that helps them understand their mistakes in a deeper way [1].

D. PERSONALIZED LEARNING WITH LLMs

In addition to providing automated feedback and debugging support, LLMs are used to create personalized learning environments. Systems such as Tutorly, which leverage LLMs to response adaptively to student learning needs, have been shown to improve student engagement and satisfaction [14]. These systems use LLMs to tailor feedback and learning materials to the specific needs and abilities of individual students, helping to close the gap between students of varying skill levels in large and diverse classrooms [4].

The use of LLMs in personalized learning contexts allows greater flexibility in instructional design, as educators can develop adaptive learning pathways that respond to student progress in real time [3]. This adaptability ensures that all students receive the support they need to succeed, regardless of their initial level of programming proficiency [13]. However, challenges remain, particularly in ensuring that LLMs do not promote excessive dependence on automated tools, which can hinder the development of independent problem solving skills [15].

III. METHOD

This review followed a systematic protocol inspired by PRISMA guidelines [21].

This review of the literature examines the use of LLMs in the assessment process in programming education and was carried out following the guidelines proposed by [21] for evidence-based software engineering. The process involved defining the research questions, selecting appropriate databases, applying inclusion and exclusion criteria, and synthesizing the results to assess the LLM-based tools used in computer programming education.

The search strategy was applied to the following academic databases: **IEEE Xplore**, **ACM Digital Library**, **Springer-Link**, and **Scopus**. The timeframe considered was from *January 2019 to December 2024*.

Inclusion Criteria:

- Peer-reviewed articles reporting applications of LLMs in programming teaching or evaluation;
- Studies including qualitative or quantitative analysis of learning outcomes, code correctness, or feedback quality;
- Articles in English.

Exclusion Criteria:

- Articles unrelated to programming education;
- Theoretical papers without application or evaluation;
- Duplicate studies across databases.

A total of 154 records were initially identified. After duplicate removal and eligibility screening, 32 studies were selected for full-text analysis. A PRISMA flow diagram [21] is used to illustrate the selection process.

Identification

TABLE 1. Number of studies per database.

Database	Search Results
ACM	72
IEEE	13
SpringerLink	22

Records identified from databases: = 154 (IEEE Xplore, ACM Digital Library, SpringerLink and Scopus)

Screening Records after duplicates removed: 127. Records screened by title and abstract: 127. Records excluded: 78

Eligibility

Full-text articles assessed: 49. Full-text articles excluded with reasons: 17

Included

Studies included in qualitative synthesis: 32

A. RESEARCH QUESTIONS

This leads to the following research questions (RQs):

RQ1 - Which LLM-based contributions are applied in the computer programming teaching evaluation processes?

This question investigates how LLMs are used in educational assessments, including automated feedback systems, code repair assistance, and personalized learning.

RQ2 - How are LLMs utilized to evaluate and provide students with programming assignment feedback?

This question examines how LLMs such as GPT-4 have been applied to automate and personalize feedback, addressing both the accuracy of assessments and the scalability in educational environments.

RQ3 - Which methodologies are employed to analyze the efficacy of LLM-based evaluation tools in improving learning outcomes?

This question sought to identify the methodological approaches used to assess the pedagogical impact of LLM.

B. SEARCH STRATEGY

A comprehensive search strategy was used, using search strings such as ‘programming education,’ ‘evaluation process,’ and ‘large language models’ across IEEE Xplore, ACM Digital Library, and SpringerLink databases [2], [8]. These keywords were chosen based on their prevalence in the literature and relevance to the research questions, as can be seen in Table 1.

C. EXCLUSION CRITERIA

To ensure the relevance and suitability of the selected articles for this review, the following exclusion criteria were applied, as can be summarized in Table 3:

1) NON-SPECIFIC AI EDUCATION TOOLS

Papers that focused broadly on AI in education without specific mention or focus on LLMs were excluded from this study. For example, studies on general AI tutoring systems and automated grading without LLM technologies were not included [12].

TABLE 2. Exclusion criteria applied to the study selection process.

Exclusion Criteria	YES	NO
Non-Specific AI Education Tools	7	32
Outdated or Irrelevant Applications	7	32
Studies Without Empirical Data	9	30
Excluded	9	30

2) OUTDATED OR IRRELEVANT APPLICATIONS

Studies published before 2019 or those using outdated AI models not classified as large language models were excluded. This ensures that the review reflects the current and relevant advancements in LLM technologies, as shown in [22], who evaluated the modern prompt engineering techniques used to improve the feedback quality in programming tasks.

3) STUDIES WITHOUT EMPIRICAL DATA

Studies that were theoretical, speculative or lacked empirical data were excluded. The review concentrates on studies with real-world applications and measurable impacts, such as the work by [8] and [13] on the PyDex system, which provides LLM-based program repair evidence that surpasses traditional methods.

D. INCLUSION CRITERIA

Inclusion criteria, as can be summarized in Table 2, were developed to ensure that only studies that focus on the use of LLM for feedback generation, program repair, and adaptive learning in programming education were included [1], [21]. Studies were included if they were as follows:

1) FOCUS ON LLMs IN PROGRAMMING EDUCATION

Only articles that specifically discuss the use of LLM in the context of programming education context were included. These include feedback generation, program repair, and personalized learning systems. For example, studies such as [13] that evaluated the ability of GPT-4 to provide feedback in large-scale programming courses were considered essential.

2) EVALUATION PROCESSES

This review prioritized studies that specifically addressed LLM-facilitated evaluation processes, such as automated feedback mechanisms, debugging assistance, and adaptive learning systems [12].

3) EMPIRICAL STUDIES WITH MEASURABLE OUTCOMES

Articles that provided empirical evidence on the effectiveness of LLM-based systems in improving learning outcomes were included [2], [3]. This category comprises studies such as [7], which explored how adaptive learning through LLM improved student engagement and learning outcomes in introductory programming courses.

TABLE 3. Inclusion criteria and number of studies included.

Inclusion Criteria	Yes	No
Focus on LLMs in Programming Education	31	8
Evaluation Processes	31	8
Empirical Studies with Measurable Outcomes	31	8
Published in Peer-Reviewed Journals and Conferences	39	0
Inclusion Criteria	31	0

4) PUBLISHED IN PEER-REVIEWED JOURNALS AND CONFERENCES

Only articles published in reputable peer-reviewed academic journals and conferences were considered to ensure that the findings were rigorous and credible.

At the end of the application of the inclusion and exclusion criteria, remained a total of 31 articles.

E. DATA EXTRACTION AND ANALYSIS

Data extraction was performed using a predefined format to ensure consistency and transparency [8]. The extracted data included the study objectives, methodologies, evaluation metrics, key findings, and implications for programming education. This process allows for a systematic comparison of LLM-based tools in different contexts, ensuring that the review covers the full range of applications in programming education [9].

A thematic synthesis approach was used to group the findings into feedback, program, and adaptive learning categories. The synthesis was designed to highlight common themes in the studies, as well as unique contributions from specific papers [3], [4]. This review aimed to evaluate the overall effectiveness of LLM-based tools in improving student learning outcomes, focusing on their potential for scalability, personalized feedback, and long-term educational impact [1], [11].

A standardized data extraction form was used for each included article to systematically capture key information. The extracted data included the following.

1) RESEARCH OBJECTIVES

What were the objectives? Is it intended to improve feedback processes, help with debugging, or personalize learning experiences using LLMs?

2) METHODOLOGY AND APPROACH

What methodology was used in the present study? Did it use experimental setups, case studies, or large-scale evaluation?

3) EVALUATION METRICS

How was LLM performance in programming education evaluated? Metrics include feedback accuracy, error correction rates, student engagement, and learning outcomes.

4) KEY FINDINGS

The main results of the study include whether LLM improved student performance, supported personalized learning, and enabled scalable feedback. For example, [14] found that the Tutorly system, powered by LLMs, improves student engagement by transforming static programming videos into interactive learning environments. In another example, [8] showed that the PyDex system, which uses Codex for program repair, is more effective than traditional repair tools in fixing syntactic and semantic bugs in Python assignments.

5) IMPLICATIONS FOR PROGRAMMING EDUCATION

What are the implications of using LLMs in programming education? This includes insights into scalability and personalized learning as discussed by [23] in their evaluation of feedback propagation to scale LLM usage in massive open online courses (MOOCs).

Each article was evaluated using the following criteria to ensure the quality of the included studies:

6) METHODOLOGY RIGOR

Studies were evaluated for the robustness of the methodology, including the clarity of the research design, the appropriateness of the evaluation metrics, and the validity of the results. For example, [13] demonstrated high rigor by comparing GPT-4 performance with open source models in multiple metrics, such as feedback accuracy and student satisfaction.

7) RELEVANCE TO THE RESEARCH QUESTIONS

Papers were included if they provided direct insight into the research questions posed by this review, particularly about the use of LLM in feedback generation, program repair, and adaptive learning. All articles classified and evaluated in this study had a Quality Assessment classified as High or Moderate.

8) DATA TRANSPARENCY

The clarity with which the data and results were presented and whether the study provided sufficient detail to allow replication or further investigation.

F. SYSTEMATIC MAPPING STUDIES IN THIS PAPER

This paper reflects a systematic mapping study that categorizes LLM applications in education programming, identifies research gaps, and provides a structured overview of empirical findings rather than conducting a meta-analysis. This approach follows the systematic mapping process described in [21], which is particularly useful in fields where research is still emerging and there are insufficient mature studies for a full systematic review.

1) HIGH-LEVEL CLASSIFICATION OF STUDIES

This paper categorizes existing research into key areas such as automated feedback, program repair, adaptive learning, and scalability. This classification aligns with the systematic

mapping approach, which aims to identify clusters and gaps in research [21].

2) EVIDENCE CLUSTERING AND RESEARCH GAPS IDENTIFICATION

The research maps primary studies based on their contributions to the assessment process, focusing on evaluation methodologies, key metrics, and pedagogical impacts. This approach reflects the purpose of systematic mapping, which is to provide an overview of available research rather than a detailed synthesis [21].

3) RESEARCH QUESTIONS ALIGNED WITH MAPPING OBJECTIVES

The research questions (RQs) in this study focus on how LLM is used in teaching evaluation, their impact, and the methodologies for evaluation. Systematic mapping studies often aim to categorize research rather than provide a deep statistical synthesis, which aligns with the objectives of this review.

4) DESCRIPTIVE SYNTHESIS INSTEAD OF A META-ANALYSIS

Instead of performing a meta-analysis, this study synthesizes findings through thematic analysis, a method commonly employed in systematic mapping reviews.

IV. RESULTS

A. RQ1 - WHICH CONTRIBUTIONS BASED ON LLMs ARE APPLIED IN THE EVALUATION PROCESSES IN COMPUTER PROGRAMMING TEACHING

In analyzing the evaluation processes in computer programming education using LLM, distinct categories emerged from the literature. These categories, which are summarized in Table 4, highlight the different ways in which LLMs are applied in programming education and their respective benefits. An in-depth discussion of these categories will be provided, followed by an analysis which category yields the most promising results.

1) AUTOMATED FEEDBACK SYSTEMS

LLMs are widely recognized for their ability to provide automated feedback on programming assignments. Traditional feedback methods often involve a delay due to manual review by instructors, especially in large courses. Automated systems powered by LLMs can provide students with instant, tailored feedback, helping them correct errors and improve their coding practices.

- Main Benefits

Automated feedback saves time for instructors, offers immediate correction of both syntactical and conceptual errors, and can scale up for large courses such as MOOCs. Studies by [23] demonstrated that GPT-4 outperforms open source models in term of feedback accuracy and pedagogical value. Feedback is provided quickly and at a level appropriate student's understand-

ing, ranging from basic syntax issues to more complex logic errors.

- **Challenges**

While feedback is immediate, there are concerns regarding over-reliance on automated systems. As noted in [19], students may become overly dependent on AI-generated solutions without developing deeper problem-solving skills. Furthermore, ensuring feedback contextualization for more complex errors or subtle learning needs remains challenging even for the most advanced LLMs.

2) ADAPTIVE LEARNING AND PERSONALIZED INSTRUCTION

LLMs also play a crucial role in adaptive learning, where educational content and feedback are tailored to the student's performance. Adaptive learning systems powered by LLM adjust difficulty level, instruction pace, and feedback type based on individual student interactions.

- **Main Benefits**

These systems are highly beneficial for personalized learning, as shown in [14] with the Tutorly system, which transforms passive programming tutorials into interactive, apprenticeship-style environments. The adaptive nature of LLM-powered systems ensures that students receive challenge at the right level, keeping them engaged while improving their understanding. [7] also demonstrated that feedback personalization through the ChatGPT increased engagement and improved learning outcomes.

- **Challenges**

One of the main issues with adaptive learning is the computational complexity and resources required to personalize learning at scale. Furthermore, while LLMs can tailor feedback, their ability to deeply understand individual learning trajectories over extended periods remains limited. The complexity of balancing learning scaffolding with challenge levels requires ongoing development.

3) PROGRAM REPAIR AND BUG FIXING ASSISTANCE

LLMs have been increasingly applied to program repair, helping students debug and correct their code. Instead of simply pointing out errors, these systems suggest possible fixes and guide students understand their mistakes.

- **Main Benefits** Tools such as PyDex, described by [8], showed that LLMs such as Codex are more effective than traditional repair tools in fixing syntactic and semantic errors. By providing immediate bug fixes along with explanations, these systems help students to better understand why their code fails and how to avoid similar issues in the future.

- **Challenges**

Similar to automated feedback, the main concern here is the student dependency on AI for bug fixing. There is the risk that students will rely on LLM suggestions without

TABLE 4. Results and discussion for RQ1 – contributions of LLMs in programming education Assessment.

Category	References
Automated Feedback Systems	[23], [13], [9]
Adaptive Learning and Instruction	[14], [4]
Program Repair and Bug Fixing	[8], [24], [11]
Prompt Engineering	[22], [8]
Others *	[14], [4]

developing deep debugging skills. The explanations provided by LLM also vary in quality, depending on the complexity of the error.

4) PROMPT ENGINEERING AND FEEDBACK OPTIMIZATION

The feedback quality provided by LLMs is highly dependent on how they are prompted. Prompt engineering has emerged as a field focused on crafting the inputs given to LLMs in a manner that maximizes output usefulness.

- **Main Benefits**

Research by [22] showed that well-written prompts significantly improve the feedback quality LLMs provide, especially in complex tasks like code generation and bug fixing. Proper prompt design can reduce ambiguity in LLM responses, ensuring that feedback is clear, relevant, and actionable.

- **Challenges**

Effective prompts require significant expertise, and poorly designed prompts can result in incomplete or inaccurate feedback. Furthermore, it can be challenging to generalize prompt engineering techniques in different educational contexts, making it necessary to continually refine and test prompts based on course-specific requirements [22].

B. RQ2 - HOW ARE LLMs USED TO EVALUATE AND PROVIDE FEEDBACK ON PROGRAMMING ASSIGNMENTS?

1) AUTOMATING CODE REVIEW AND FEEDBACK GENERATION WITH LLMs

The literature has widely explored the LLM-use to evaluate and provide feedback on programming assignments. These models have been used to automate code review and feedback generation, as demonstrated by GPT [3]. In addition, LLMs have been used to provide next-step hints in programming exercises, fostering interactive learning [10]. LLM-powered tools have also been applied to program repair and automated feedback generation, helping identify and correct errors [24]. Specifically, these models have effectively fixed evaluated bugs in introductory Python programming assignments, highlighting their potential as pedagogical tools [8].

2) PROVIDING PERSONALIZED FEEDBACK FOR PROGRAMMING WITH LLMs

Large language models (LLMs) have also been utilized to provide personalized feedback for programming assignments and enhance the learning experience. These models are

integrated into introductory computer science courses (CS1) to provide personalized debugging assistance and learning feedback to address the specific needs of individual students [2]. Furthermore, LLMs such as GPT-4 are used to offer tiered formative feedback, adapting the level of detail and guidance based on the proficiency levels of students, thus ensuring a more customized and effective educational approach [12].

3) SCALABLE FEEDBACK IN PROGRAMMING USING LLMs

Large language models (LLMs) have proven valuable in providing scalable feedback for programming assignments and addressing the challenges of supporting a large number of learners. To ensure that the feedback remains accurate and reliable on the scale, guardrails are implemented to maintain consistency and quality [9]. LLMs are also used to expand feedback and support for software development lifecycle (SDLC) assignments, streamlining the evaluation process [25]. Furthermore, these models facilitate the propagation of feedback in massive open online courses (MOOCs), allowing efficient support for large-scale programming courses and ensuring that students receive timely actionable insights [23].

4) IMPROVING FEEDBACK ACCURACY IN PROGRAMMING

Efforts to improve the accuracy and feedback success of large language models (LLMs) in programming assignments have focused on refining their implementation and evaluation. Prompt engineering has emerged as a key strategy for improving the accuracy and usefulness of feedback, particularly in Python programming tasks [26]. LLMs are also used to generate detailed and meaningful feedback for programming exercises, contributing to better student learning outcomes [6]. Specifically, models such as GPT-4 have been used to produce feedback for various programming tasks, highlighting their potential in educational settings [27]. Furthermore, research has evaluated the performance of LLM in providing feedback for Parsons problems, which are widely used to assess programming skills, further highlighting their applicability in diverse pedagogical contexts [28].

5) CONVERSATIONAL AND INTERACTIVE SUPPORT FOR PROGRAMMING WITH LLMs

Large language models are also being utilized to provide conversational and interactive support for programming assignments, enhancing student engagement and understanding. LLM-powered conversational tools offer real-time assistance and guidance, making it easier for learners to navigate challenges in their coding tasks [29]. Research has explored how platforms such as ChatGPT deliver feedback and support text-based programming, demonstrating the potential of conversational agents to foster a more interactive learning experience [5]. In addition, innovative applications include the transformation of tutorial programming videos into interactive learning environments, where students can

TABLE 5. References grouped by LLM applications in programming education.

Group	References
Automating Code Review and Feedback Generation with LLMs	[3], [27], [24], [8]
Providing Personalized Feedback for Programming with LLMs	[27], [12]
Scalable Feedback in Programming Using LLMs	[27], [25], [13]
Improving Feedback Accuracy in Programming	[26], [6], [27], [28]
Conversational and Interactive Support for Programming with LLMs	[29], [5], [14]
Formative Feedback and Analysis of Help Requests Using LLMs	[15], [17]
Applying Data Analytics and LLM Tools for Feedback in Education	[16], [30], [18]
Autonomous Support and Visual Learning with LLMs	[31]
Others *	[14], [4]

receive real-time feedback, bridging the gap between passive content consumption and active problem solving [14].

6) FORMATIVE FEEDBACK AND ANALYSIS OF HELP REQUESTS USING LLMs

LLMs are increasingly being used to generate formative feedback and analyze student help requests in programming assignments, offering valuable support for learners. By providing formative feedback, these models help students identify areas for improvement and guide them toward solutions during the learning process [15]. Furthermore, research has focused on evaluating the accuracy of LLM in responding to beginner help requests, ensuring that the support offered is aligned with learners' needs and contributes effectively to their understanding of programming concepts [17].

7) APPLYING DATA ANALYTICS AND LLM TOOLS FOR FEEDBACK IN EDUCATION

The Data analytics and LLM tools integration is advancing feedback effectiveness in programming education, particularly to address the needs of struggling students. Learning analytics combined with LLM feedback interventions help identify and support learners who face difficulties, providing targeted help to improve their performance [16]. LLMs have also been evaluated for their ability to generate feedback in complex tasks such as concurrent programming assignments, demonstrating their utility in handling advanced topics [30]. Furthermore, a structured query language has been proposed to improve the interaction with LLMs, allowing more precise and efficient feedback generation, which further enriches the learning experience [18].

8) AUTONOMOUS SUPPORT AND VISUAL LEARNING WITH LLMs

LLMs are increasingly used to facilitate autonomous learning, particularly in the realm of visual programming. By offering real-time feedback, these models allow students to navigate independently and improve their programming skills in a dynamic, visually interactive environment [31].

C. RQ3 - WHICH METHODOLOGIES ARE EMPLOYED TO ASSESS THE LLM-BASED EVALUATION TOOLS EFFECTIVENESS IN IMPROVING LEARNING OUTCOMES?

The methodologies used to assess the effectiveness of LLM-based evaluation tools in improving learning outcomes fall into several categories. Several methodologies were used in this study, which are summarized in Table 6:

1) EMPIRICAL EVALUATIONS

Many studies have adopted empirical evaluations to assess the performance of LLM-based tools by measuring outcomes such as student learning gains, engagement, and feedback accuracy. These studies often involve pre- and post-tests to quantify improvements in student knowledge and task performance.

For example, [2] and [12] employed empirical methodologies by integrating LLM-based feedback systems into CS1 courses and assesses learning improvements through formative and summative assessments. Similarly, [6] and [26] evaluated feedback quality by tracking student progress over time and analyzing its impact on learning outcomes.

2) CASE STUDIES

Case studies are another common methodology used to evaluate LLM-based tools, especially in smaller controlled environments where researchers can observe individual student interactions with LLM systems. For instance, [31] conducted a case study in which an LLM-powered tool was applied to learning of visual programming for children, assessing improvements in real-time feedback and task completion rates. Similarly, [24] used case studies to assess the LLM-powered program repair tools success such as InferFix, tracking how students benefited from automated debugging.

3) COMPARATIVE ANALYSES

Several studies have compared the performance of LLM-based tools with traditional teaching and evaluation methods. For example, [10] compared the effectiveness of LLM-generated hints with traditional instructor feedback in introductory programming assignments, measuring engagement, time to task completion, and overall learning improvements. Reference [8] compared the PyDex performance, an LLM-based bug repair system, against traditional debugging tools, and founding that LLM systems were more efficient in fixing errors and supporting learning.

4) MIXED-METHODS APPROACHES

Studies such as [20] and [25] employ mixed methods approaches by combining quantitative metrics (e.g. task accuracy, error correction rates, student performance) with qualitative feedback from students and instructors to holistically assess the LLM impact on learning. These studies often use surveys, interviews, and classroom observations along

TABLE 6. Results and discussion for RQ3 – methodologies to assess LLM-based evaluation Tools.

Methodology	References
Empirical Evaluations	[2], [6], [13]
Case Studies	[25], [32]
Comparative Analyses	[8], [11], [25]
Mixed-Methods Approaches	[21], [26]
Controlled Experiments	[28], [29]
Longitudinal Studies	[2], [15]
Others *	[15], [4]

with objective performance data to gain a comprehensive understanding of how LLM-based tools influence learning.

5) CONTROLLED EXPERIMENTS

Controlled experimental setups were used to systematically assess the impact of the LLM-based feedback tools on learning. For example, [28] tested LLM models to solve Parsons problems, measured the accuracy and correctness of the generated solutions and analyze the results through controlled testing environments. Similarly, [27] evaluated GPT-4's ability to provide feedback for programming exercises by analyzing both feedback accuracy and students' ability to improve based on that feedback.

Table 6 summarizes the methodologies for evaluating LLM-based evaluation tools.

V. TECHNICAL DISCUSSION ON LLM AND THEIR ROLE IN TEACHING EVALUATION

In this research, we identified the underlying LLM mechanisms and their impact on teaching evaluation, focusing on prompt engineering, feedback propagation, and adaptive learning techniques. Their effectiveness in the teaching evaluation process originate from advances in natural language processing (NLP), transformer architectures, and self-supervised learning [6], [32].

A. PROMPT ENGINEERING AND RESPONSE OPTIMIZATION

Prompt engineering is a commonly used technique to improve the reliability and relevance of LLM-generated outputs. LLM relies on well-structured instructions to understand the context and produce responses aligned with pedagogical objectives [6], [12]. Effective prompt engineering strategies include:

- zero-shot and few-shot prompting: enhancing model performance by providing examples within the prompt to guide response generation [2].
- iterative refinement: dynamically adjust prompts based on student responses to ensure tailored feedback [14].
- Task decomposition: breaking down complex coding problems into smaller subtasks, allowing llms to provide step-by-step guidance [30].

These techniques contribute to increasing accurate and context-aware feedback, reducing ambiguity, and improving student comprehension.

B. AUTOMATED FEEDBACK GENERATION AND PROPAGATION

LLM improves the assessment process by providing instant feedback on student submissions and facilitating the rapid identification of errors and misconceptions. Feedback propagation allows models to refine their evaluations based on previous interactions, thereby increasing their effectiveness over time [8], [27]. common applications include the following:

- syntax and logic error detection: identifying and explaining errors in the code structure and logical implementation [11].
- adaptive feedback loops: modifying explanations based on student engagement and prior misunderstandings [20].
- progressive hinting: gradually guiding students toward correct solutions rather than providing direct answers [33].

These capabilities enable LLM to function as scalable assessment tools, reducing instructor workload while maintaining feedback quality.

C. ADAPTIVE LEARNING AND PERSONALIZED EVALUATION

One of the key advantages of LLM is its ability to adapt to responses based on student proficiency levels. Traditional assessment methods often lack personalization, while LLM utilizes contextual memory and interaction patterns to offer customized assessments [9], [10]. Key features include:

- competency-based evaluation: adjusting the complexity of questions and feedback based on student performance trends [16].
- Natural language explanations: transforming complex programming concepts into simpler conversational explanations [5].
- Dynamic reinforcement: identify recurring student mistakes and adapt exercises to address specific learning gaps [19].

This personalization improves student engagement and retention of programming concepts.

D. LIMITATIONS AND CHALLENGES OF LLM-BASED ASSESSMENT

Despite their potential, LLM-based evaluation systems present several challenges:

- hallucinations and misinformation: LLM can generate incorrect feedback, necessitating human verification [17], [18].
- Overreliance on automation: excessive use of LLM-generated feedback may hinder students' ability to develop independent problem-solving skills [12].
- bias and fairness: pre-trained models may exhibit biases in feedback patterns, requiring continuous validation and bias mitigation strategies [33].

Addressing these limitations is critical to ensure that LLM supports, rather than replaces, human instructors in the evaluation process.

1) THE OVER-RELIANCE IN AUTOMATED SYSTEMS

While LLM offers significant advantages in automating feedback and assessment in programming education, concerns have been raised regarding the potential over-reliance on these systems. Excessive dependence on automated feedback can hinder students' ability to develop critical thinking and problem solving skills, particularly in debugging and code optimization tasks [12], [19].

2) THE IMPACT OF AUTOMATION DEPENDENCE ON LEARNING OUTCOMES

Several studies indicate that while LLM-generated feedback accelerates the debugging process, it may also reduce students' engagement in understanding errors at a deeper level. Reference [17] demonstrated that students who relied exclusively on automated debugging suggestions without instructor intervention exhibited lower retention rates in problem-solving techniques than those who participated in guided troubleshooting. Similarly, [18] found that while LLM-assisted assessments improved code correctness, they often led students to accept solutions without critically analyzing alternative approaches.

3) STRATEGIES TO MITIGATE OVER-RELIANCE ON AUTOMATION

To address these challenges, researchers have proposed hybrid assessment frameworks that combine automated feedback and structured human oversight. Key strategies include:

- Progressive disclosure of hints: rather than providing direct answers, LLMs should guide students through incremental hints, encouraging them to analyze the problem independently before revealing the full solution [33].
- Adaptive challenge levels: dynamically adjusting the difficulty of programming exercises based on student proficiency to ensure that they develop problem-solving resilience [14].
- Self-explanation prompts: students to articulate their reasoning when incorporating LLM-generated suggestions to reinforce their conceptual understanding [5].
- combination of peer and instructor review: Integrating human review mechanisms where students can compare automated feedback with peer discussions and instructor insights to validate learning [30].

E. PERFORMANCE EVALUATION FOR LLM-BASED IN STUDENT FEEDBACK

Evaluating LLM effectiveness in the student feedback process requires a structured approach that considers multiple performance dimensions, including accuracy, response time,

and student satisfaction. Previous studies have explored these aspects to measure the pedagogical value of llm-generated feedback in programming education [8], [11], [14]. Based on the literature, we classified LLM evaluation in feedback generation into three key metrics:

1) ACCURACY AND RELEVANCE OF FEEDBACK

The accuracy of the llm-generated feedback is crucial for ensuring that students receive meaningful and pedagogically sound guidance. Studies have assessed accuracy through the following:

- Precision and recall: Evaluation of the proportion of correctly identified errors and appropriate corrections compared with expert ground-truth feedback [11], [27].
- Semantic coherence: analyze whether feedback explanations align with the best practices of accepted programming [12].
- Error categorization: distinguishing between syntax errors, logic errors, and conceptual misunderstandings [2], [3].

These measures ensure that the feedback generated by LLM is reliable and supports effective learning outcomes.

2) RESPONSE TIME AND SCALABILITY

Response efficiency is essential for large-scale educational environments, such as massive open online courses (MOOCs). evaluating llms based on response time includes the following:

- Latency: measuring the time it takes for the llm to generate feedback after receiving student input [9], [10].
- Scalability: analyzing the model performance under high concurrent request loads while maintaining accuracy [21].

3) STUDENT PERCEPTION AND ENGAGEMENT

In addition to technical accuracy and efficiency, student engagement and perception of llm-generated feedback play a crucial role in determining its effectiveness. The qualitative and quantitative methods used in prior studies include the following:

- feedback adoption rate: assessing how often students incorporate llm-generated feedback into their corrections [5], [16].
- learning outcome improvement: measuring progress through pre- and post-assessment comparisons in llm-assisted learning environments [19].
- student satisfaction surveys: analyzing subjective feedback from students on usability, trust, and perceived effectiveness of llm feedback [33].

4) CHALLENGES IN LLM-BASED FEEDBACK EVALUATION

Although llm-generated feedback provides benefits in scalability and real-time assistance, challenges remain in ensuring high-quality, personalized, and pedagogically sound feedback. Key concerns include:

- hallucination and misinformation: instances where llms generate misleading or incorrect explanations, necessitating human oversight [17].
- overreliance on automation: the risk that students become overly dependent on llm assistance instead of developing independent debugging skills [12], [19].
- adaptability limitations: difficulty in providing fully personalized feedback that considers students' prior knowledge and learning history [14].

VI. CONCLUSION

This systematic literature review analyzed 32 studies focused on the application of LLMs in computer programming education. The findings reveal that LLMs are primarily used for feedback automation, debugging assistance, and interactive tutoring. These models demonstrate potential to enhance engagement, streamline feedback, and support learners with varying skill levels.

This review demonstrated that Large Language Models (LLMs) offer significant potential to improve learning outcomes in programming education, particularly through feedback automation, debugging assistance, and personalized learning. Reviews of studies have consistently shown that LLM can improve student participation, reduce the time required to complete the task, and improve the precision of feedback provided to learners [8], [10]. Specifically, tools such as PyDex and InferFix have been shown to be effective in identifying and repairing syntactic and semantic bugs, thereby accelerating the learning process [11].

A. INFLUENCE AND INNOVATION

- For Industry: This review highlights how LLM streamline code assessment, is crucial for large-scale training programs and technical interviews. Their ability to automate and personalize feedback can reduce training time and improve programmer onboarding efficiency.
- For Academia: Academically, this review provides a framework for educators to integrate LLMs into curricula, improve feedback scalability, and foster self-directed learning. This review also recommends hybrid LLM-human classification systems to support critical thinking development and limit overreliance on automated solutions.
- Uniqueness: Unlike previous studies, this review emphasizes the challenges of scalability and proposes prompt engineering techniques as a vital improvement area. In addition, longitudinal studies of LLMs' long-term impacts on programming skills, a gap not addressed in other reviews.

However, although the initial results are promising, the limitations of LLM must be acknowledged. As highlighted in [13], LLMs perform well in simpler tasks but struggle with more complex high-order reasoning problems. This presents a challenge for their widespread application

in advanced programming tasks, where human expertise remains crucial. Moreover, [15] caution that overreliance on automated feedback can hinder the development of students' independent problem-solving skills, raising concerns about the sustainability of LLM-driven learning environments.

Despite these challenges, the scalability and adaptability of LLMs make them an invaluable tool in large-scale educational settings. The ability of LLM to provide personalized real-time feedback to hundreds or even thousands of students simultaneously is a clear advantage over traditional methods [4], [13]. In addition, the integration of LLMs into educational platforms such as Tutorly has shown that they can support individualized learning pathways and tailor feedback to the specific needs and proficiency levels of each student [14].

Future research should explore hybrid human-AI feedback frameworks, investigate instructional strategies that best complement LLMs, and evaluate their impact on deep learning outcomes beyond code correctness. Furthermore, more longitudinal studies are needed to evaluate the long-term impact of LLM-driven feedback on student learning, particularly in terms of fostering independence and reducing the overreliance on AI-generated solutions [1].

B. FINAL REMARKS

Despite the growing body of research on LLM applications in education, existing reviews focus primarily on their general capabilities or isolated case studies in specific educational contexts [2], [6], [12]. However, a comprehensive synthesis that systematically evaluates their role in programming education, particularly in teaching assessment and feedback mechanisms, remains an open research gap. This study provides a structured review that advances the current understanding of LLM applications in teaching evaluation through the following contributions:

- Systematic classification of LLM-based evaluation methodologies: Previous work has explored the use of LLM in programming education but often lacks a structured taxonomy of evaluation approaches. This study categorizes existing methods based on the scope, level of automation, and pedagogical alignment of the evaluation, thereby providing a clear roadmap for future research [3], [11], [27].
- Quantitative and qualitative assessment metrics: While previous studies have mentioned LLM effectiveness in education, few have proposed concrete, standardized evaluation metrics. Our review consolidates empirical studies and introduces a framework that includes accuracy, response latency, scalability, and student engagement as key performance indicators [8], [14], [16].
- Critical discussion on LLM limitations in automated assessment: existing research tends to emphasize the benefits of LLMs without addressing key challenges such as hallucinations, overreliance on automation,

and ethical concerns. Our study presents a balanced perspective, incorporating recent findings on LLM reliability, bias, and the need for human oversight in educational settings [17], [18], [19].

REFERENCES

- [1] J. Smith and L. Williams, "Automated grading and feedback tools for programming education: A systematic literature review," *J. Comput. Higher Educ.*, vol. 32, no. 4, pp. 315–335, 2023.
- [2] A. Vadaparty, D. Zingaro, D. H. Smith, M. Padala, C. Alvarado, J. G. Benario, and L. Porter, "CS1-LLM: Integrating LLMs into CS1 instruction," *ACM Inroads*, pp. 297–303, Jul. 2024.
- [3] D. Lee, "A GPT-based code review system for programming language learning," in *Proc. Int. Conf. Learn. Teaching Comput. Eng.*, Jun. 2024.
- [4] M. Abolnejadian, S. Alipour, and K. Taeb, "Leveraging ChatGPT for adaptive learning through personalized prompt-based instruction: A CS1 education case study," in *Proc. Extended Abstr. CHI Conf. Human Factors Comput. Syst.*, May 2024, pp. 1–8.
- [5] M. Wieser, K. Schöffmann, D. Stefanics, A. Bollin, and S. Pasterk, "Investigating the role of ChatGPT in supporting text-based programming education for students and teachers," *ACM Trans. Comput. Educ.*, pp. 40–53, Jan. 2023.
- [6] S. Jacobs and S. Jaschke, "Evaluating the application of large language models to generate feedback in programming education," *Comput. Educ.*, pp. 1–5, May 2024.
- [7] M. Abolnejadian et al., "Leveraging ChatGPT for adaptive learning through personalized prompt-based instruction: A cs1 education perspective," *ACM Inroads*, 2024.
- [8] J. Zhang, J. P. Cambronero, S. Gulwani, V. Le, R. Piskac, G. Soares, and G. Verbruggen, "PyDex: Repairing bugs in introductory Python assignments using LLMs," in *Proc. ACM Program. Lang.*, vol. 8, Apr. 2024, pp. 1100–1124.
- [9] M. Liffiton, B. E. Sheese, J. Šavelka, and P. Denny, "CodeHelp: Using large language models with guardrails for scalable support in programming classes," *IEEE Trans. Learn. Technol.*, pp. 1–11, Nov. 2023.
- [10] J. Roest, "Automated next-step hint generation for introductory programming using large language models," *ACM Trans. Comput. Educ.*, 2024.
- [11] H. Jin, S. H. Lee, H. Shin, and J. Kim, "Teach AI how to code: Using large language models as teachable agents for programming education," *IEEE Trans. Learn. Technol.*, pp. 1–28, May 2024.
- [12] H. Nguyen and V. Allan, "Using GPT-4 to provide tiered, formative code feedback," in *Proc. ACM Tech. Symp. Comput. Sci. Educ. (SIGCSE)*, 2024, pp. 958–964.
- [13] C. Koutchme and A. Hellas, "Propagating large language models programming feedback," in *Proc. IEEE Conf. Educ. Data Mining (EDM)*, 2024, pp. 366–370.
- [14] X. Li et al., "Tutorly: Turning programming videos into apprenticeship learning environments with LLMs," *ACM Trans. Comput. Educ.*, 2024.
- [15] N. Kiesler, D. Lohr, and H. Keuning, "Exploring the potential of large language models to generate formative programming feedback," *ACM Trans. Comput. Educ.*, Oct. 2023.
- [16] P. Utamachant, C. Anutariya, and S. Pongnumkul, "I-ntervene: Applying an evidence-based learning analytics intervention to support computer programming instruction," *Smart Learn. Environments*, vol. 10, no. 1, Jul. 2023.
- [17] A. Hellas, J. Leinonen, S. Sarsa, C. Koutchme, L. Kujanpää, and J. Sorva, "Exploring the responses of large language models to beginner Programmers' help requests," *J. Educ. Comput. Res.*, pp. 93–105, Aug. 2023.
- [18] L. Beurer-Kellner, M. Fischer, and M. Vechev, "Prompting is programming: A query language for large language models," in *Proc. ACM Symp. Theory Comput. (STOC)*, vol. 7, Jun. 2023, pp. 1946–1969.
- [19] B. E. Sheese, M. Liffiton, J. Šavelka, and P. Denny, "Patterns of student help-seeking when using a large language model-powered programming assistant," in *Proc. ACM Conf. Learn. Scale*, Jan. 2024, pp. 49–57.
- [20] E. Brown and M. Clarke, "Scaling student feedback in software engineering massive open online courses," in *Proc. Int. Conf. Softw. Eng.*, 2023, pp. 98–105.
- [21] B. Kitchenham and S. Charters, "Guidelines for performing systematic literature reviews in software engineering," Dept. Comput. Sci., Univ. Durham, Durham, U.K., Tech. Rep. EBSE-2007-01, 2007.

- [22] S. Gao, X.-C. Wen, C. Gao, W. Wang, H. Zhang, and M. R. Lyu, "What makes good in-context demonstrations for code intelligence tasks with LLMs?" in *Proc. ACM Trans. Intell. Syst. Technol.*, Jan. 2024.
- [23] K. Koutchme and A. Hellas, "Propagating large language models programming feedback," *ACM Trans. Comput. Educ.*, 2024.
- [24] M. Jin, S. Shahriar, M. Tufano, X. Shi, S. Lu, N. Sundaresan, and A. Svyatkovskiy, "InferFix: End-to-end program repair with LLMs," in *Proc. 31st ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Nov. 2023, pp. 1646–1656.
- [25] T. Wang et al., "DevCoach: Supporting students in learning the software development life cycle at scale with generative models," *IEEE Access*, 2024.
- [26] T. Wang, N. Zhou, and Z. Chen, "Enhancing computer programming education with LLMs: A study on effective prompt engineering for Python code generation," 2024, *arXiv:2407.05437*.
- [27] I. Azaiz, N. Kiesler, and S. Strickroth, "Feedback-generation for programming exercises with GPT-4," *IEEE Access*, pp. 31–37, 2024.
- [28] D. Reeves et al., "Evaluating the performance of code generation models for solving parsons problems with small prompt variations," in *Proc. ACM Trans. Comput. Educ.*, 2023.
- [29] I. Zamfirescu-Pereira et al., "Conversational programming with LLM-powered interactive support in an introductory computer science course," *IEEE Trans. Educ.*, 2024.
- [30] I. Estévez-Ayres, P. Callejo, M. Á. Hombrados-Herrera, C. Alario-Hoyos, and C. Delgado Kloos, "Evaluation of LLM tools for feedback generation in a course on concurrent programming," *Int. J. Artif. Intell. Edu.*, vol. 35, no. 2, pp. 774–790, Jun. 2025.
- [31] Z. Chen et al., "Chatscratch: An AI-augmented system toward autonomous visual programming learning for children aged," *J. Educ. Comput. Res.*, 2024.
- [32] T. Brown, B. Mann, N. Ryder, and M. Subbiah, "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 1877–1901.
- [33] M. Virvou and G. A. Tsihrintzis, "Is ChatGPT beneficial to education? A holistic evaluation framework based on intelligent tutoring systems," in *Proc. 14th Int. Conf. Inf., Intell., Syst. Appl. (IISA)*, Jul. 2023, pp. 1–6.



ANDRE FABIANO PEREIRA received the bachelor's degree in computer science from the Federal University of Maranhão, Brazil, and the master's degree in computer science from the Federal University of Amazonas, Brazil. He is currently pursuing the Ph.D. degree with the Centro de Estudos e Sistemas Avançados do Recife, Brazil. He is the CEO of the Everest Technology and Innovation Institute, Amazonia, Brazil. His research interests include software engineering and large language models.



RAFAEL FERREIRA MELLO (Member, IEEE) received the Ph.D. degree in computer science. He is currently a Professor with UFRPE and the Centro de Estudos e Sistemas Avançados do Recife, Brazil, where he holds the position of Chair of Education Technology Adoption and Data Analytics. In the last few years, he has supported the adoption of learning analytics techniques and tools in several institutions in Brazil. One recognition of his contributions is that he is the leading Researcher in two national-level projects, funded by the Brazilian Ministry of Education. He has published in leading international journals and conference proceedings in the fields of his research. His research interests include learning analytics, generative AI, and natural language processing. He also served as an Assistant Editor and a Reviewer for several journals and conferences, including the International Learning Analytics and Knowledge Conference, *Journal of Learning Analytics, Computers and Education, British Journal of Educational Technology*, and *The Internet and Higher Education*.

• • •

Coordenação de Aperfeiçoamento de Pessoal de Nível Superior (CAPES) - ROR identifier: 00x0ma614